

STRIPE PROJECT

8. Global Data Architecture

D1. Data Architecture Diagram

Nicolas Pichon - AIA RNCP 38777 / BC02 / D1 : Data Architecture Diagram / v10 - 2026/10/13.

D1.1. Contexte

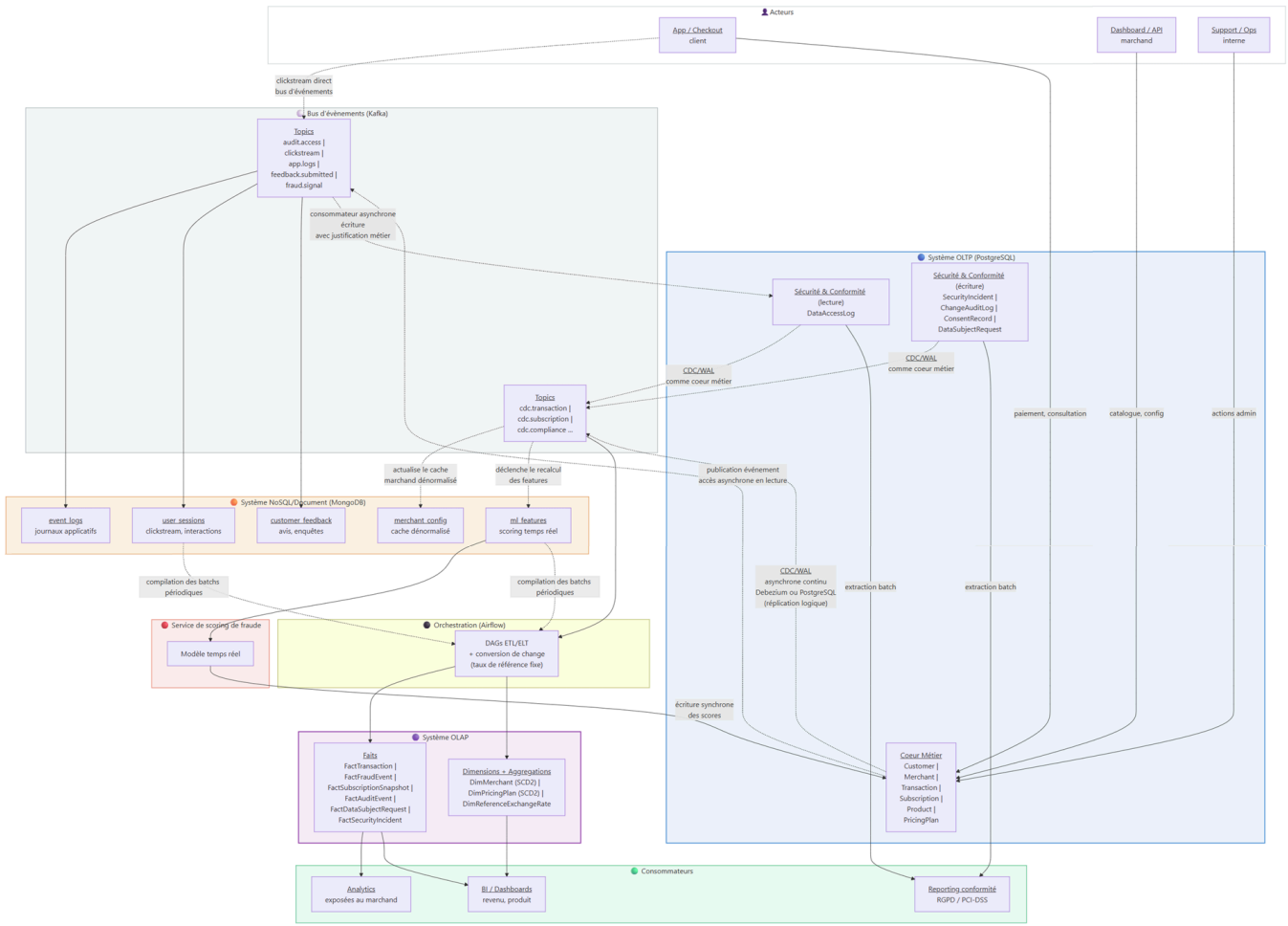
Ce document présente les grands principes de l'architecture de données du *business case* [R0], montrant l'intégration des systèmes OLTP [D2], OLAP [D3] et NoSQL [D4] :

- flux de données,
- pipelines,
- et modèles de données.

Trois grands principes structurent l'architecture :

1. *Le chemin transactionnel ne doit jamais être ralenti* par des processus de sécurité, de conformité, d'analytique ou de détection de fraude. Les flux de données qui partent du système transactionnel vers les autres systèmes sont donc asynchrones.
2. *Eviter les triggers synchrones pour répliquer les changements. Utiliser les systèmes CDC (Change Data Capture) du système OLTP
3. *Le taux de change est fixe dans la base analytique (actualisable périodiquement, annuellement par exemple).

D1.2. Diagramme d'architecture globale des données



D1.3. Lecture des flux de données

Flux	Mécanisme	Justification
Client / Marchand → OLTP	Écriture synchrone directe	Chemin transactionnel, ACID requis (BR1)
OLTP coeur → Topic CDC	CDC (Debezium ou réplication logique PostgreSQL)	Même principe que <code>ChangeAuditLog</code> (Annexe 1.C) : jamais de trigger synchrone sur <code>Transaction</code>
OLTP coeur → Topic applicatif (accès)	Publication d'événement, asynchrone	Un <code>SELECT</code> sur <code>OLTP_CORE</code> ne laisse aucune trace dans le WAL - rien à capter en CDC pour créer la ligne (Annexe 1.C)
Topic applicatif → <code>DataAccessLog</code> (création)	Consommateur asynchrone	Le consommateur porte le contexte métier (justification) que seule l'application connaît au moment de l'accès
<code>DataAccessLog</code> / <code>ChangeAuditLog</code> / <code>ConsentRecord</code> / <code>DataSubjectRequest</code> / <code>SecurityIncident</code> → topic CDC	CDC (même mécanisme que <code>OLTP_CORE</code>)	Une fois la ligne insérée dans l'OLTP (par le consommateur pour <code>DataAccessLog</code> , normalement pour les quatre autres), le CDC la réplique comme n'importe quelle autre table
Topic applicatif → NoSQL	Ingestion continue	Absorbe logs, clickstream, interactions (DS3) - schéma flexible, volumétrie non bornée
Topic CDC → <code>merchant_config</code> (NoSQL)	Consommateur asynchrone	Cache dénormalisé actualisé à chaque changement <code>Merchant / Customer / Transaction</code> (§D4.8 [D4]) ; évite d'interroger OLTP à chaque événement applicatif
Topic CDC → <code>ml_features</code> (NoSQL)	Déclenchement asynchrone	Le recalcul des features suit le même flux CDC que <code>merchant_config</code> ; pas de mécanisme distinct
NoSQL features → service de scoring	Lecture temps réel	Alimente le modèle de fraude en continu
Service de scoring → OLTP	Écriture synchrone ciblée sur <code>FraudScore</code>	Table découplée de <code>Transaction</code> (annexe 1.C) : n'ajoute aucun verrou sur le chemin critique
Topic CDC + NoSQL → Airflow → OLAP	Batch ETL/ELT planifié	Conversion de change appliquée ici, avec taux de référence figé - jamais recalculée côté OLTP
OLAP → BI / marchand	Requêtes analytiques	Découplé du chemin transactionnel, aucun impact sur la latence de paiement
Conformité OLTP → reporting	Extraction batch	Alimente les rapports RGPD/PCI-DSS sans exposer l'OLTP directement aux outils de reporting

D1.4. Sujets non couverts

La sélection des technologies et des outils managés sous-jacents (*MongoDB Atlas*, self-hosted, ou service compatible de type *DocumentDB*), tout comme le service *Kafka* ou l'entrepôt OLAP, est une décision d'infrastructure et d'hébergement hors du périmètre de l'architecture de données.

La stratégie de réplication/failover propre à *OLTP* est détaillée séparément (cf. [D2-C]).

Références

Ressources

- [R0] [Stripe Business Case](#)

Livrables

- [D2] [OLTP Entity-Relation Diagrams](#)
- [D3] [OLAP System Schema Design](#)
- [D4] [NoSQL Data Model](#)

Annexes

- [D2-C] [OLTP Supporting Notes](#)